



Variable in C/C++

We may classify variables into two categories:

- A. Local variables.
- B. Global Variables.



Local variables

- ▶ These variables are declared inside the function within any code block, which mean that local variables are not known outside their own code block.

Ex:–

```
void func1(void)
{
    int x;
    x=10;
}

void func2(void)
{
    int x;
    x=-199;
}
```



Global variable

- ▶ They are known throughout the entire program and may be used by any piece of code. They are created and declaring outside any function.

Ex:-

```
#include <iostream>
int x=1;
void main ()
{
}
```



2.6 Arithmetic

- ▶ Most programs perform arithmetic calculations.
- ▶ Figure 2.9 summarizes the C++ **arithmetic operators**.
- ▶ The **asterisk** (*) indicates multiplication.
- ▶ The **percent sign** (%) is the **modulus** operator that will be discussed shortly.
 - C++ provides the **modulus operator**, %, that yields the remainder after integer division.
 - The modulus operator can be used only with integer operands.
- ▶ The arithmetic operators in Fig. 2.9 are all binary operators.
- ▶ **Integer division** (i.e., where both the numerator and the denominator are integers) yields an integer quotient.
 - Any fractional part in integer division is discarded (i.e., **truncated**)—no rounding occurs.



C++ operation	C++ arithmetic operator	Algebraic expression	C++ expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm or $b \cdot m$	<code>b * m</code>
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Modulus	%	$r \bmod s$	<code>r % s</code>

Fig. 2.9 | Arithmetic operators.



2.6 Arithmetic (cont.)

- ▶ Arithmetic expressions in C++ must be entered into the computer in **straight-line form**.
- ▶ Expressions such as “a divided by b” must be written as a / b , so that all constants, variables and operators appear in a straight line.
- ▶ Parentheses are used in C++ expressions in the same manner as in algebraic expressions.
- ▶ For example, to multiply a times the quantity $b + c$ we write $a * (b + c)$.



2.6 Arithmetic (cont.)

- ▶ C++ applies the operators in arithmetic expressions in a precise sequence determined by the following **rules of operator precedence**, which are generally the same as those followed in algebra.



Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are <i>nested</i> , such as in the expression $a * (b + c / d + e)$, the expression in the <i>innermost</i> pair is evaluated first. [Caution: If you have an expression such as $(a + b) * (c - d)$ in which two sets of parentheses are not nested, but appear “on the same level,” the C++ Standard does <i>not</i> specify the order in which these parenthesized subexpressions will be evaluated.]
*	Multiplication	Evaluated second. If there are several, they’re evaluated left to right.
/	Division	
%	Modulus	
+	Addition	Evaluated last. If there are several, they’re evaluated left to right.
-	Subtraction	

Fig. 2.10 | Precedence of arithmetic operators.



2.6 Arithmetic (cont.)

- ▶ There is no arithmetic operator for exponentiation in C++, so x^2 is represented as $x * x$.
- ▶ Figure 2.11 illustrates the order in which the operators in a second-degree polynomial are applied.
- ▶ As in algebra, it's acceptable to place unnecessary parentheses in an expression to make the expression clearer.
- ▶ These are called **redundant parentheses**.

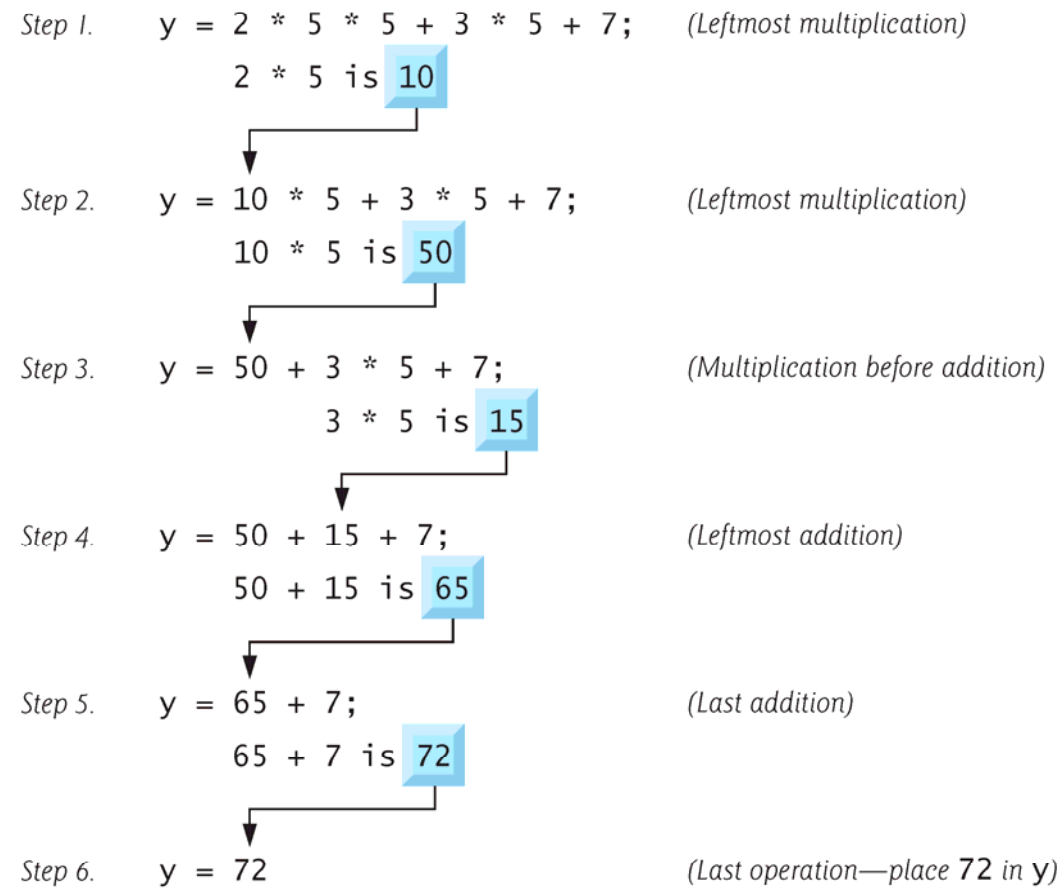


Fig. 2.11 | Order in which a second-degree polynomial is evaluated.



Bitwise operators

- ▶ Bitwise operators modify variables considering the bit patterns that represent the values they store

Bitwise operator	
&	AND
	OR
^	Exclusive or(xor)
~	Not
>>	Shift right
<<	Shift left



Bitwise operators(cont.)

- ▶ Bitwise operators can be used only on type int and char. Type float, double, longdouble, void or other more complex types cannot be used.



Bitwise operators(cont.)

Ex1:-

x=13; (=0 0 0 0 1 1 0 1)

y=6; (=0 0 0 0 0 1 1 0)

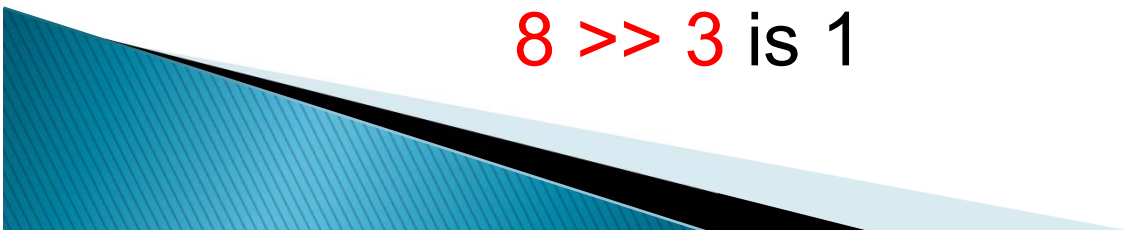
z=x & y; z=4;

Ex2-

2 << 4 is 32

and-

8 >> 3 is 1





2.7 Decision Making: Equality and Relational Operators

- ▶ The **if statement** allows a program to take alternative action based on whether a **condition** is true or false.
- ▶ If the condition is true, the statement in the body of the **if** statement is executed.
- ▶ If the condition is false, the body statement is not executed.
- ▶ Conditions in **if** statements can be formed by using the **equality operators** and **relational operators** summarized in Fig. 2.12.
- ▶ The relational operators all have the same level of precedence and associate left to right.
- ▶ The equality operators both have the same level of precedence, which is lower than that of the relational operators, and associate left to right.



Algebraic relational or equality operator	C++ relational or equality operator	Sample C++ condition	Meaning of C++ condition
<i>Relational operators</i>			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
<i>Equality operators</i>			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Fig. 2.12 | Relational and equality operators.



2.7 Decision Making: Equality and Relational Operators (cont.)

- ▶ The following example uses six **if** statements to compare two numbers input by the user.
- ▶ If the condition in any of these **if** statements is satisfied, the output statement as-sociated with that **if** statement is executed.
- ▶ Figure 2.13 shows the program and the input/output dialogs of three sample executions.



```
1 // Fig. 2.13: fig02_13.cpp
2 // Comparing integers using if statements, relational operators
3 // and equality operators.
4 #include <iostream> // allows program to perform input and output
5
6 using std::cout; // program uses cout
7 using std::cin; // program uses cin
8 using std::endl; // program uses endl
9
10 // function main begins program execution
11 int main()
12 {
13     int number1 = 0; // first integer to compare (initialized to 0)
14     int number2 = 0; // second integer to compare (initialized to 0)
15
16     cout << "Enter two integers to compare: "; // prompt user for data
17     cin >> number1 >> number2; // read two integers from user
18
19     if ( number1 == number2 )
20         cout << number1 << " == " << number2 << endl;
21 }
```

Fig. 2.13 | Comparing integers using if statements, relational operators and equality operators. (Part I of 3.)



```
22  if ( number1 != number2 )
23      cout << number1 << " != " << number2 << endl;
24
25  if ( number1 < number2 )
26      cout << number1 << " < " << number2 << endl;
27
28  if ( number1 > number2 )
29      cout << number1 << " > " << number2 << endl;
30
31  if ( number1 <= number2 )
32      cout << number1 << " <= " << number2 << endl;
33
34  if ( number1 >= number2 )
35      cout << number1 << " >= " << number2 << endl;
36  } // end function main
```

```
Enter two integers to compare: 3 7
3 != 7
3 < 7
3 <= 7
```

Fig. 2.13 | Comparing integers using `if` statements, relational operators and equality operators. (Part 2 of 3.)



```
Enter two integers to compare: 22 12
22 != 12
22 > 12
22 >= 12
```

```
Enter two integers to compare: 7 7
7 == 7
7 <= 7
7 >= 7
```

Fig. 2.13 | Comparing integers using if statements, relational operators and equality operators. (Part 3 of 3.)



2.7 Decision Making: Equality and Relational Operators (cont.)

- ▶ **using declarations** that eliminate the need to repeat the `std::` prefix as we did in earlier programs.
- ▶ Once we insert these **using** declarations, we can write `cout` instead of `std::cout`, `cin` instead of `std::cin` and `endl` instead of `std::endl`, respectively, in the remainder of the program.
- ▶ Many programmers prefer to use the declaration
`using namespace std;`
which enables a program to use all the names in any standard C++ header file (such as `<iostream>`) that a program might include.
- ▶ From this point forward in the book, we'll use the preceding declaration in our programs.



2.7 Decision Making: Equality and Relational Operators (cont.)

- ▶ Each **if** statement in Fig. 2.13 has a single statement in its body and each body statement is indented.
- ▶ In Chapter 4 we show how to specify **if** statements with multiple-statement bodies (by enclosing the body statements in a pair of braces, { }, creating what's called a **compound statement** or a **block**).



Common Programming Error 2.5

Placing a semicolon immediately after the right parenthesis after the condition in an `if` statement is often a logic error (although not a syntax error). The semicolon causes the body of the `if` statement to be empty, so the `if` statement performs no action, regardless of whether or not its condition is true. Worse yet, the original body statement of the `if` statement now becomes a statement in sequence with the `if` statement and always executes, often causing the program to produce incorrect results.



2.7 Decision Making: Equality and Relational Operators (cont.)

- ▶ Statements may be split over several lines and may be spaced according to your preferences.
- ▶ It's a syntax error to split identifiers, strings (such as "hello") and constants (such as the number 1000) over several lines.



- ▶ The Operator ! is the C++ operator to perform the Boolean operation NOT, it has only one operand, located at its right, and the only thing that it does is to inverse the value of it, producing false if its operand is true and true if its false
- ▶ operand is false. Basically, it returns the opposite Boolean value of evaluating its operand. For example:

```
!(5 == 5)    // evaluates to false because the expression at its right (5 == 5) is true.  
!(6 <= 4)    // evaluates to true because (6 <= 4) would be false.
```




Increase and decrease ($++$, $--$)

- ▶ Shortening even more some expressions, the increase operator ($++$) and the decrease operator ($--$) increase or reduce by one the value stored in a variable. They are equivalent to $+=1$ and to $-=1$, respectively. Thus:

$a++$;

$a+=1$;

$a=a+1$;

are all equivalent in its functionality: the three of them increase by one the value of a .



- ▶ Although in simple expressions like `a++` or `++a` both have exactly the same meaning, in other expressions in which the result of the increase or decrease operation is evaluated as a value in an outer expression they may have an important difference in their meaning: In the case that the increase operator is used as a prefix (`++a`) the value is increased before the result of the expression is evaluated and therefore the increased value is considered in the outer expression; in case that it is used as a suffix (`a++`) the value stored in `a` is increased after being evaluated and therefore the value stored before the increase.

Example 1	Example 2
<pre>B=3; A=++B; // A contains 4, B contains 4</pre>	<pre>B=3; A=B++; // A contains 3, B contains 4</pre>



Return Statement

- ▶ In C++, main() return an integer type value to the operating system. Therefore, every main() in C++ should end with return 0 statement, otherwise a warning or an error might occur.

```
#include <iostream>
using namespace std;
int main( )
{
    float number1,number2, sum;
    cin>>number1;
    cin>>number2;
    sum=number1+number2;
    return 0;
}
```



- ▶ Or we can use void main (), this doesn't end with return 0 and no warning or an error will occur.

```
#include <iostream>  
using namespace std;
```

```
void main( )  
{  
float number1,number2, sum;  
cin>>number1;  
cin>>number2;  
sum=number1+number2;  
}
```



Basic Input/Output

- ▶ C++ uses a convenient abstraction called *streams* to perform input and output operations in sequential media such as the screen or the keyboard. A stream is an object where a program can either insert or extract characters to/from it. We do not really need to care about many specifications about the physical media associated with the stream – we only need to know it will accept or provide characters sequentially.
- ▶ The standard C++ library includes the header file `iostream`, where the standard input and output stream objects are declared.



Standard Output (cout)

- ▶ The standard output of a program is the screen, and the C++ stream object defined to access it is cout.

```
cout << "Output sentence";    // prints Output sentence on screen
cout << 120;                  // prints number 120 on screen
cout << x;                    // prints the content of x on screen
```

```
cout << "Hello";              // prints Hello
cout << Hello;                 // prints the content of Hello variable
```



Standard Output (cout)

- ▶ The insertion operator (<<) may be used more than once in a single statement:

```
cout << "Hello, " << "I am " << "a C++ statement";
```

- ▶ This last statement would print the message **Hello, I am a C++ statement** on the screen.
- ▶ The utility of repeating the insertion operator (<<) is demonstrated when we want to print out a combination of variables and constants or more than one variable:

```
cout << "Hello, I am " << age << " years old and my zipcode is " << zipcode;
```



Standard Input (cin)

- ▶ The standard input device is usually the keyboard. Handling the standard input in C++ is done by applying the overloaded operator of extraction (>>) on the cin stream. The operator must be followed by the variable that will store the data that is going to be extracted from the stream. For example:

```
int age;  
cin >> age;
```




- ▶ You can also use cin to request more than one datum input from the user:

`cin >> a >> b;`

is equivalent to:

`cin >> a;`

`cin >> b;`